

Vận dụng tư tưởng quy chuyển để giải các bài toán dãy số trong tin học

Dư Lâm Vận

Giáo viên hướng dẫn: Trần Đình
Trường Trung học số 1 Phúc Châu, lớp 12-9

Tháng 11 năm 2007

Nguồn gốc: Using reduction to solve sequence problems in informatics

I0I-China-Candidate-Team-Papers/2008/Day2/6-Yu-Linyun/sequence-problems-reduction.pdf

Trích yếu

Dãy số là một nhánh quan trọng của toán học và có phạm vi ứng dụng rộng. Từ xưa đến nay đã có nhiều công trình kinh điển nghiên cứu các vấn đề về dãy số. Bài viết này thử vận dụng kiến thức toán học, đứng từ góc nhìn giải bài toán bằng máy tính, kết hợp với các bài toán dãy số trong thi tin học để phân tích, nghiên cứu và thảo luận. Mục tiêu là nêu ra các phương pháp tư duy và kỹ thuật giải bài toán dãy số, đồng thời truyền đạt một thông điệp: giải bài toán dãy số không nên rập khuôn; cần dám đổi mới, liên hệ các kiến thức đã học và phối hợp nhiều phương pháp.

Nội dung được chia thành bốn chương. Chương đầu giới thiệu các phương pháp toán học cần dùng khi giải bài toán dãy số, làm nền cho những chương sau. Chương thứ hai chọn hai ví dụ tiêu biểu mà phía sau có ẩn một tính chất tuần hoàn đẹp. Khai thác hợp lý tính tuần hoàn của dãy số, kết hợp với các công cụ đã nêu, có thể giảm mạnh độ phức tạp thời gian và gợi ý cách giải cho những bài toán tương tự. Chương thứ ba chọn bài Count của NOI 2007 ngày 2 và bài 1396 trên Ural làm ví dụ, trên cơ sở khai thác tính tuần hoàn để tiếp tục giới thiệu các cách quy chuyển khác: đào sâu bản chất bài toán, tìm công thức số hạng tổng quát, xét bài toán theo từng giai đoạn. Chương cuối phân tích lại vai trò của dãy số trong thi tin học và nhấn mạnh các tư tưởng phương pháp đã dùng.

Từ khóa: dãy số, quy chuyển, thuật toán Euclid, lũy thừa nhanh, nhân ma trận, tính tuần hoàn, phương pháp đa dạng.

Lời nói đầu

Máy tính đã dần đi vào đời sống con người, mạng máy tính ngày càng phổ biến và cuộc cách mạng thông tin đã đến. Tin học, với tư cách một ngành mới, ngày càng được các nước coi trọng. Bước sang thế kỷ mới, cùng với giáo dục toàn diện và cải cách chương trình, giáo dục tin học ở Trung Quốc đã trở thành nội dung bắt buộc từ bậc tiểu học và kéo dài đến đại học.

Học tin học cần nắm chắc các môn nền tảng như toán học và vật lý. Ngược lại, thông qua học tập, thực hành và ứng dụng tin học, ta rèn được năng lực tư duy logic chặt chẽ, thúc đẩy tinh thần sáng tạo và nâng cao tố chất tổng hợp. Trong quá trình học tin học, ta thường gặp nhiều bài toán dãy số. Lịch sử

ngiên cứu dãy số của nhân loại rất lâu đời: cùng với khái niệm số tự nhiên, phân số và bốn phép toán, con người đã bắt đầu nghiên cứu dãy số để phục vụ sản xuất và đời sống. Trong lịch sử toán học thế giới, Trung Quốc, Babylon, Hy Lạp cổ đại, Ai Cập và Ấn Độ đều từng nghiên cứu cấp số. Các tác phẩm toán học cổ Trung Hoa từng nêu ví dụ tính cấp số cộng

$$a + (a + b) + (a + 2b) + (a + 3b) + \dots + [a + (n - 1)b]$$

và cấp số nhân

$$a + aq + aq^2 + aq^3 + \dots + aq^{n-1}.$$

Điều đó cho thấy nghiên cứu về cấp số đã có đóng góp từ rất sớm. Câu nói cổ “Thái cực sinh lưỡng nghi, lưỡng nghi sinh tứ tượng, tứ tượng sinh bát quái” thực ra cũng đã hàm chứa tư tưởng của cấp số nhân.

Vì kiến thức về dãy số có vai trò quan trọng trong giải quyết vấn đề thực tế, các nước chưa từng ngừng nghiên cứu nó. Ở Trung Quốc, dãy số được xem là nền tảng cho giải tích và là chất liệu tốt để rèn năng lực học sinh, nên được đưa vào chương trình toán trung học phổ thông.

Những học sinh từng tham gia các kỳ thi toán tiểu học hẳn đã gặp các câu hỏi kiểu “tìm quy luật để điền số”. Trong các kỳ thi tin học, cũng có rất nhiều bài thú vị về dãy số hoặc liên quan đến dãy số. Bài viết này chọn một số bài tiêu biểu, giải theo đặc trưng riêng của từng bài, từ đó rút ra các quy luật chung với hy vọng gợi mở thêm suy nghĩ.

Giải bài toán dãy số vừa đòi hỏi kiến thức toán học phong phú, vừa đòi hỏi tư duy mở, biết vận dụng đầy đủ tư tưởng quy chuyển, mạnh dạn phỏng đoán và liên tục đào sâu bản chất bài toán.

Quy chuyển là chuyển hóa và quy về. Khi giải bài toán, ta thường biến bài toán A chưa giải được, thông qua một quá trình chuyển hóa nào đó, thành bài toán B đã biết cách giải hoặc dễ giải hơn, rồi từ lời giải của B quay lại tìm lời giải của A . Bản chất của phương pháp này là biến vấn đề mới thành kiến thức cũ đã nắm được, sau đó hiểu và giải quyết vấn đề mới. Ba yếu tố của quy chuyển là đối tượng quy chuyển, mục tiêu quy chuyển và con đường quy chuyển.

Các kiến thức toán học rộng hơn đi kèm bài viết này đã được đặt trong bản dịch riêng `sources/ioi/2008/math-related-knowledge-yu-linyun.tex`; tài liệu này chỉ nhắc lại những phần trực tiếp được dùng trong lập luận.

1 Kiến thức chuẩn bị

1.1 Nhân ma trận

Cho hai ma trận $A = (a_{ij})_{n \times m}$ và $B = (b_{ij})_{m \times t}$. Phép nhân ma trận $A \cdot B$ cho ma trận

$$C = A \cdot B = (c_{ij})_{n \times t}, \quad c_{ij} = \sum_{k=1}^m a_{ik}b_{kj}.$$

Từ công thức trên, độ phức tạp của phép nhân là $O(nmt)$. Nếu các ma trận đều có kích thước $N \times N$, độ phức tạp là $O(N^3)$. Về lý thuyết có các thuật toán nhanh hơn $O(N^3)$, nhưng hằng số lớn khiến chúng thường không hiệu quả hơn trong cài đặt thi đấu thông thường.

Phép nhân ma trận không giao hoán, nhưng có tính kết hợp. Vì vậy, để tính A^M với A là ma trận vuông, có thể dùng tư tưởng nhân đôi. Ký hiệu E là ma trận đơn vị.

```
function MatrixPower(A, M)
    result <- E
    x <- A
    y <- M
    while y > 0 do
```

```

        if y is odd then
            result <- result * x
        end if
        x <- x * x
        y <- y div 2
    end while
    return result
end function

```

1.2 Thuật toán Euclid

1.2.1 Thuật toán cơ bản

Ước chung lớn nhất của hai số là ước lớn nhất cùng chia hết cả hai số đó. Ký hiệu ước chung lớn nhất của a, b là $\gcd(a, b)$. Thuật toán Euclid dựa trên hai tính chất sau:

- a) Ước chung lớn nhất của a và một bội của a là a .
- b) Nếu q và r lần lượt là thương và số dư khi chia a cho b , tức $a = bq + r$, thì

$$\gcd(a, b) = \gcd(b, r).$$

Tính chất thứ nhất suy ra trực tiếp từ định nghĩa. Với tính chất thứ hai, đặt $x = \gcd(a, b)$ và $y = \gcd(b, r)$. Vì $x \mid a$ và $x \mid b$, nên $x \mid (a - bq)$, tức $x \mid r$, do đó $x \mid y$. Ngược lại, $y \mid b$ và $y \mid r$ nên $y \mid (bq + r)$, tức $y \mid a$, do đó $y \mid x$. Vì $x \mid y$ và $y \mid x$, suy ra $x = y$.

```

function gcd(a, b)
    if b = 0 then
        return a
    else
        return gcd(b, a mod b)
    end if
end function

```

1.2.2 Độ phức tạp

Trong mỗi hai bước liên tiếp, đối số lớn giảm ít nhất một nửa. Cụ thể, nếu $b \leq a/2$ thì $r < b \leq a/2$; nếu $b > a/2$ thì $r = a - b < a/2$. Vì thế số lần gọi $\gcd(x, y)$ nhiều nhất là $O(\log_2 a)$, và độ phức tạp là $O(\log_2 a)$.

1.2.3 Mở rộng

Cho a, b , cần tìm một cặp (x, y) sao cho

$$ax + by = \gcd(a, b).$$

Gọi q, r là thương và số dư khi chia a cho b , tức $a = bq + r$. Nếu đã có x, y sao cho

$$by + xr = \gcd(b, r),$$

thì

$$by + x(a - qb) = b(y - qx) + ax = \gcd(b, r) = \gcd(a, b).$$

Do đó $(x, y - qx)$ là một nghiệm của $ax + by = \gcd(a, b)$.

```

function extended_gcd(a, b)
  if b = 0 then
    x <- 1
    y <- 0
    return a, x, y
  else
    d, y, x <- extended_gcd(b, a mod b)
    y <- y - (a div b) * x
    return d, x, y
  end if
end function

```

1.3 Tính nhanh số hạng thứ N của dãy truy hồi tuyến tính hệ số hằng

1.3.1 Cấp số nhân

Từ công thức tổng quát của cấp số nhân

$$a_n = a_1 q^{n-1},$$

muốn tính nhanh a_n ta chỉ cần tính nhanh q^{n-1} . Ta khai triển $n - 1$ theo nhị phân và dùng các giá trị $q, q^2, q^4, \dots, q^{2^k}$.

```

function Power(q, n)
  result <- 1
  x <- q
  y <- n
  while y > 0 do
    if y is odd then
      result <- result * x
    end if
    x <- x * x
    y <- y div 2
  end while
  return result
end function

```

Như vậy có thể tính số hạng thứ N của cấp số nhân trong $O(\log_2 N)$ thời gian.

1.3.2 Dãy Fibonacci tổng quát

Xét dãy Fibonacci tổng quát với $F_1 = a, F_2 = b$ và $F_i = F_{i-1} + F_{i-2}$. Các số hạng đầu là

$$F_3 = a + b, \quad F_4 = a + 2b, \quad F_5 = 2a + 3b, \quad \dots$$

Mỗi số hạng đều là tổng của một số bản sao của a và một số bản sao của b .

Ta biểu diễn quan hệ truy hồi bằng ma trận:

$$\begin{pmatrix} F_{i-1} \\ F_i \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} F_{i-2} \\ F_{i-1} \end{pmatrix}.$$

Suy ra

$$\begin{pmatrix} F_{n-1} \\ F_n \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}^{n-1} \begin{pmatrix} F_1 \\ F_2 \end{pmatrix}.$$

Vì lũy thừa ma trận trên tính được trong $O(\log_2 N)$ thời gian, ta cũng tính được F_N trong $O(\log_2 N)$ thời gian.

1.3.3 Dãy truy hồi tuyến tính hệ số hằng

Với dãy tổng quát hơn, nếu biết k số hạng đầu của $\{a_n\}$ và biết quan hệ giữa mỗi số hạng với k số hạng trước nó, ta có thể lập một ma trận $k \times 1$ lưu trạng thái và một ma trận $k \times k$ chuyển trạng thái. Nhân lũy thừa thứ $n - k$ của ma trận chuyển với trạng thái ban đầu sẽ cho số hạng thứ n .

Cụ thể, nếu

$$a_{n+k} = c_1 a_{n+k-1} + c_2 a_{n+k-2} + \dots + c_k a_n,$$

ma trận chuyển có dạng

$$\begin{pmatrix} 0 & 1 & 0 & \dots & 0 \\ 0 & 0 & 1 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & 1 \\ c_1 & c_2 & c_3 & \dots & c_k \end{pmatrix}.$$

Độ phức tạp của thuật toán là

$$O(\log_2(n - k)) \cdot O(\text{nhân ma trận}) = O(k^3 \log_2 n).$$

2 Khai thác hợp lý tính tuần hoàn của dãy số

Ngoài các dãy truy hồi tuyến tính hệ số hằng, những dãy truy hồi phi tuyến đặc biệt cũng thường xuất hiện trong thi tin học. Với một phương trình truy hồi phi tuyến tổng quát, không có phương pháp vạn năng; bài toán cũng không chỉ là tìm công thức số hạng tổng quát, vì đôi khi công thức ấy rất khó viết tường minh. Thường ta cần xác định một số tính chất của số hạng.

Mỗi dãy số có đặc trưng riêng. Các đặc trưng ấy thường thể hiện qua tính tuần hoàn hoặc tính theo giai đoạn, và rất có ích cho lời giải. Chương này dùng hai ví dụ để trình bày cách phân tích sâu, khai thác hợp lý tính tuần hoàn và quy chuyển thành công.

2.1 Ví dụ 1: Dispute

Mô tả bài toán. Dãy F_n thỏa mãn

$$F_0 = 0, \quad F_n = g(n, F_{n-1}) \quad (n \geq 1),$$

trong đó

$$g(x, y) = ((y - 1)x^5 + x^3 - xy + 3x + 7y) \bmod 9973.$$

Cho n , hãy tính F_n . Bài toán xuất phát từ Ural 1309; để tiện tính toán, dãy có số hạng thứ 0.

Dữ liệu.

$$0 \leq n \leq 100000000.$$

Phân tích. Vì n có thể tới 10^8 , truy hồi trực tiếp là khó chấp nhận. Quan sát công thức truy hồi, ta thấy nó không tuyến tính, nên không thể áp dụng nhân ma trận một cách đơn giản. Các số hạng đầu

$$3, 93, 2818, 968, 2132, \dots$$

cũng chưa cho thấy quy luật rõ ràng.

Thuật toán thứ nhất. Khi phân tích ở mọi hướng đều vướng, dùng vét cạn có kiểm soát vẫn là một cách đáng cân nhắc. Truy hồi trực tiếp không chịu nổi vì miền của n quá lớn. Nếu thu hẹp n về một phạm vi chấp nhận được thì sao?

Ta có thể viết một chương trình thường để tính các F_x , rồi lưu lại mọi giá trị có chỉ số là bội của 100000 vào một danh sách. Khi xử lý một truy vấn, ta bắt đầu truy hồi từ $F_{\lfloor n/100000 \rfloor \cdot 100000}$; số bước còn lại chỉ là $O(100000)$.

Nhìn từ góc độ thực dụng, đây là một lời giải có thể chấp nhận. Tuy nhiên, nó chưa tận dụng được vẻ đẹp của chính dãy số.

Thuật toán thứ hai. Thuật toán thứ nhất thành công vì tiền xử lý đã giảm số lần truy hồi. Nhưng tiền xử lý không phải con đường duy nhất để giảm số lần truy hồi.

Nhắc lại phần trước: nếu một dãy là truy hồi tuyến tính hệ số hằng, ta có thể dùng nhân ma trận để tính nhanh số hạng thứ N . Trong bài này, khó khăn nằm ở chỗ công thức truy hồi không tuyến tính hệ số hằng. Nhưng bản thân dãy không tuyến tính hệ số hằng không có nghĩa là mọi dãy con của nó đều như vậy. Có thể các phần tử cách nhau 2 vị trí, 3 vị trí, hoặc nhiều hơn, lại có quan hệ tuyến tính hệ số hằng. Nếu tìm được một chu kỳ k thích hợp sao cho các phần tử cách nhau k có quan hệ tuyến tính, ta sẽ khai thác được nó.

Công thức

$$g(x, y) = ((y - 1)x^5 + x^3 - xy + 3x + 7y) \bmod 9973$$

có hai điểm đáng chú ý:

- a) Trong mọi hạng tử, số mũ của y chỉ là 0 hoặc 1. Nếu coi x là hằng số, công thức tuyến tính theo y .
- b) Cuối công thức có phép lấy modulo theo số nguyên tố 9973.

Đặt $M = 9973$. Khi đó g_n tuyến tính theo F_{n-1} , và $g(x, y) = g(x \bmod M, y)$. Với những chỉ số cùng lớp dư modulo M , tồn tại hai hằng số A, B sao cho

$$F_{x+M} \equiv AF_x + B \pmod{M}.$$

Nói cách khác, trong bước nhảy M , dãy có tính tuần hoàn về dạng truy hồi. Ta có thể tính A, B trong $O(M)$ thời gian, sau đó tính F_{kM} và cuối cùng tính F_N .

Chứng minh. Đặt

$$U_i = (j^5 - j + 7) \bmod M, \quad V_i = (-j^5 + j^3 + 3j) \bmod M,$$

trong đó $j = i \bmod M + 1$. Khi đó

$$F_{i+1} = U_i F_i + V_i.$$

Với mọi K ,

$$F_{KM+1} \equiv U_{KM+1} F_{KM} + V_{KM+1} \pmod{M},$$

$$F_{KM+2} \equiv U_{KM+2} F_{KM+1} + V_{KM+2} \pmod{M},$$

và cứ thế đến $F_{(K+1)M}$. Thay liên tiếp các biểu thức vào nhau, ta được

$$F_{(K+1)M} \equiv AF_{KM} + B \pmod{M},$$

trong đó

$$A = \prod_{i=KM+1}^{(K+1)M} U_i,$$

$$B = \sum_{i=KM+1}^{(K+1)M} \left(V_i \prod_{j=i+1}^{(K+1)M} U_j \right).$$

Với hai giá trị khác nhau k_1, k_2 ,

$$U_{k_1M+i} = U_i = U_{k_2M+i}, \quad V_{k_1M+i} = V_i = V_{k_2M+i}$$

với mọi $i \in [1, M]$. Do đó A, B là hằng số, không phụ thuộc vào K .

Thuật toán này tính A, B trong $O(M)$, tính F_{KM} trong $O(K)$ tức $O(N \operatorname{div} M)$, rồi tính F_N trong $O(1)$. Tổng độ phức tạp là $O(N \operatorname{div} M)$.

Tối ưu tiếp. Trong bài này, $O(N \operatorname{div} M)$ đã ở mức $O(M)$, nhưng nếu miền của N lớn hơn nữa thì vẫn chưa đủ. Vì dãy ở bước nhảy M có quan hệ truy hồi tuyến tính hệ số hằng, ta tận dụng lại phương pháp tính nhanh.

Định nghĩa dãy mới $C_N = F_{NM}$. Khi đó

$$C_0 = 0, \quad C_1 = AC_0 + B \equiv B \pmod{M},$$

$$C_2 = AC_1 + B \equiv B(A + 1) \pmod{M},$$

$$C_3 = AC_2 + B \equiv B(A^2 + A + 1) \pmod{M},$$

và tổng quát

$$C_N \equiv B(A^{N-1} + A^{N-2} + \dots + 1) \pmod{M}.$$

Theo công thức tổng cấp số nhân,

$$C_N \equiv B \frac{1 - A^N}{1 - A} \pmod{M},$$

trong bài này $A \neq 1$.

Nếu tính trực tiếp biểu thức rồi mới lấy modulo, ta sẽ gặp số rất lớn. Vì $M = 9973$ là số nguyên tố, ta tính trước

$$x \equiv (1 - A^N)B \pmod{M}$$

bằng lũy thừa nhanh, rồi tìm $k \in [0, M - 1]$ sao cho

$$k(1 - A) \equiv x \pmod{M}.$$

Có thể duyệt k , hoặc dùng Euclid mở rộng trong $O(\log_2 M)$. Cụ thể, giải

$$k(1 - A) + tM = 1,$$

rồi $kx \bmod M$ là kết quả cần tìm.

Lý do cách này hợp lệ là M nguyên tố. Nếu có hai số khác nhau $k_1, k_2 \in [0, M - 1]$ cùng thỏa mãn, thì

$$k_1(1 - A) \equiv k_2(1 - A) \pmod{M},$$

hay

$$(1 - A)(k_1 - k_2) \equiv 0 \pmod{M}.$$

Vì $|1 - A| < M$, $1 - A \neq 0$ và $|k_1 - k_2| < M$, suy ra $k_1 = k_2$, mâu thuẫn. Do đó mỗi $x \in [0, M - 1]$ tương ứng đúng một k .

Đến đây bài toán được giải trọn vẹn với độ phức tạp

$$O(M + \log_2(N \operatorname{div} M)).$$

2.2 Ví dụ 2: Dãy Abs

Mô tả bài toán. Dãy F_n thỏa mãn

$$F_1 = A, \quad 1 \leq A \leq 10^{18},$$

$$F_2 = B, \quad 1 \leq B \leq 10^{18},$$

$$F_n = |F_{n-1} - F_{n-2}| \quad (n > 2).$$

Cho n , hãy tính F_n . Đây là một bài toán kinh điển.

Dữ liệu.

$$3 \leq n \leq 10^{18}.$$

Phân tích. Với kích thước dữ liệu như trên, ta không thể sinh n số hạng đầu. Dấu trị tuyệt đối làm dạng của công thức truy hồi thay đổi theo thời điểm: có lúc là $F_n = F_{n-1} - F_{n-2}$, có lúc là $F_n = F_{n-2} - F_{n-1}$. Vì vậy không thể dùng nhân ma trận trực tiếp.

Thay vì nhìn mãi vào công thức, ta thử chọn vài cặp A, B và quan sát:

$$A = 0, B = 1 : \quad 0, 1, 1, 0, 1, 1, 0, 1, 1, \dots$$

$$A = 1, B = 1 : \quad 1, 1, 0, 1, 1, 0, 1, 1, 0, \dots$$

$$A = 25, B = 5 : \quad 25, 5, 20, 15, 5, 10, 5, 5, 0, 5, 5, 0, \dots$$

$$A = 65, B = 26 : \quad 65, 26, 39, 13, 26, 13, 13, 0, 13, 13, 0, \dots$$

$$A = 23, B = 7 : \quad 23, 7, 16, 9, 7, 2, 5, 3, 2, 1, 1, 0, 1, 1, 0, \dots$$

Dễ thấy mọi dãy cuối cùng đều xuất hiện chu kỳ độ dài 3. Số khác 0 trong chu kỳ, gọi là x , lần lượt là 1, 5, 13, 1, Liên hệ với A, B , ta thấy

$$1 = \gcd(1, 1), \quad 5 = \gcd(25, 5), \quad 13 = \gcd(65, 26), \quad 1 = \gcd(23, 7).$$

Trường hợp $A = 0, B = 1$ là đặc biệt; nếu bỏ số hạng đầu thì có thể xem như $A = 1, B = 1$. Phỏng đoán tự nhiên là

$$x = \gcd(A, B).$$

Chứng minh. Trước hết chứng minh bằng quy nạp rằng mọi phần tử của dãy đều là bội của $t = \gcd(A, B)$.

Khi $n = 1, 2$, $F_1 = A$ và $F_2 = B$ nên kết luận đúng. Giả sử kết luận đúng với $n = k - 1, k$, tức

$$F_{k-1} = ut, \quad F_k = vt, \quad u, v \in \mathbb{N}^*.$$

Khi đó

$$F_{k+1} = |F_k - F_{k-1}| = |ut - vt| = |u - v|t,$$

nên F_{k+1} vẫn là bội của t . Do đó kết luận đúng với mọi n .

Tiếp theo, dãy nhất định sẽ có lúc xuất hiện hai số hạng liên tiếp cùng bằng xt . Trước khi điều đó xảy ra, xu hướng của dãy là giảm. Thật vậy,

$$F_{k+3} < \max(F_{k+1}, F_{k+2}), \quad F_{k+4} < \max(F_{k+2}, F_{k+3}),$$

suy ra

$$F_{k+4} < \max(F_{k+1}, F_{k+2})$$

và do đó

$$\max(F_{k+3}, F_{k+4}) < \max(F_{k+1}, F_{k+2}).$$

Giá trị nhỏ nhất của phần tử trong dãy là 0, nên quá trình giảm về sau phải dừng; vì vậy nhất định xuất hiện xt, xt .

Cuối cùng, chứng minh $x = 1$ bằng phản chứng. Nếu $x \neq 1$, mọi phần tử đều là bội của xt , mà $xt > t$. Khi đó t không thể là ước chung lớn nhất của A, B , mâu thuẫn. Do đó số khác 0 trong chu kỳ cuối cùng chính là $\gcd(A, B)$.

Kết luận này đem lại điều gì? Nhìn lại quá trình chứng minh, ta thấy nó rất giống thuật toán Euclid tìm ước chung lớn nhất. Vì vậy các đoạn đầu của dãy cũng phải liên hệ với quá trình Euclid.

Chia dãy thành nhiều giai đoạn. Để tiện, nếu $A \leq B$ thì bỏ số hạng đầu, biến thành cặp bắt đầu $B, B - A$, nhờ đó mỗi giai đoạn đều bắt đầu bằng dạng $x > y$. Ngoại trừ giai đoạn cuối đã vào chu kỳ, mỗi giai đoạn bắt đầu bằng x, y và kết thúc bằng $y, x \bmod y$.

Chi tiết một giai đoạn như sau. Đặt $\Delta = x - y$. Dãy phát triển dưới dạng

$$x, x - \Delta, \Delta, x - 2\Delta, x - 3\Delta, \Delta, x - 4\Delta, \dots$$

tức gồm nhiều khối 3 phần tử kết thúc bằng Δ . Từ đó ta có thuật toán: $f(x, y, \text{left})$ biểu diễn giá trị ở vị trí left khi hai số hạng đầu hiện tại là x, y . Đáp án là $f(A, B, N)$.

```
function f(x, y, left)
  if left = 1 then return x
  if left = 2 then return y
  if x <= y then
    return f(y, y - x, left - 1)
  end if

  delta <- x - y
  num <- x div delta
  if num is even then
    num <- num - 1
  end if
  len <- num + 1 + (num + 1) div 2

  if left > len then
    return f(x - delta * num, delta, left - len + 2)
  else
    if left mod 3 = 0 then
      return delta
    else
      return x - delta * (left - left div 3 - 1)
    end if
  end if
end function
```

Độ phức tạp phân tích tương tự thuật toán Euclid, là $O(\log_2 N)$.

2.3 Nhận xét

Ở ví dụ Dispute, ta tìm được tính tuần hoàn trong công thức truy hồi, từ đó quy về dãy truy hồi tuyến tính hệ số hằng và kết hợp lũy thừa nhanh. Ở ví dụ dãy Abs, ta nhận ra dãy cuối cùng có chu kỳ, rồi liên hệ

với thuật toán Euclid, chia bài toán thành giai đoạn để giải hiệu quả. Cả hai đều tìm được điểm đột phá từ tính tuần hoàn; thiếu nó, phân tích sâu gần như không thể bắt đầu.

3 Quy chuyển bằng phương pháp đa dạng

Các bài trong thi tin học ngày nay ngày càng thiên về kiểm tra năng lực tổng hợp thay vì chỉ áp dụng khuôn mẫu. Một bài thường lồng nhiều mô hình, và bài toán dãy số cũng không ngoại lệ. Vì vậy cần phân tích sâu hơn, bóc tách từng lớp, khai thác đầy đủ kiến thức đã học.

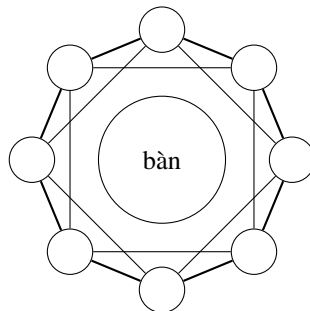
3.1 Ví dụ 3: Đếm cây khung

Mô tả bài toán. Gần đây, Tiểu Đồng có bước tiến lớn trong việc tính số cây khung của đồ thị vô hướng liên thông. Cậu phát hiện:

- Chu trình n đỉnh có n cây khung.
- Đồ thị đầy đủ n đỉnh có n^{n-2} cây khung.

Hai phát hiện này khiến cậu rất hứng thú và muốn tính số cây khung của nhiều loại đồ thị khác.

Một hôm, trong buổi họp mặt, mọi người ngồi quanh một bàn tròn lớn. Nếu xem mỗi bạn là một đỉnh và nối hai bạn ngồi cạnh nhau, ta được một chu trình. Vì Tiểu Đồng đã quá quen với chu trình, cậu thay đổi đồ thị: ngoài các bạn ngồi cạnh nhau, cậu còn nối các cặp cách nhau một ghế. Nói cách khác, hai đỉnh có khoảng cách 1 hoặc 2 trên vòng tròn thì có cạnh nối.



Hình 3.1: tám đỉnh quanh bàn tròn; hai đỉnh cách nhau 1 hoặc 2 trên vòng đều được nối.

Cậu nhớ một phương pháp chung để tính số cây khung của đồ thị bất kỳ: xây dựng ma trận $A = (a_{ij})_{n \times n}$ với

$$a_{ij} = \begin{cases} d_i, & i = j, \\ -1, & i \text{ và } j \text{ có cạnh nối,} \\ 0, & \text{ngược lại,} \end{cases}$$

trong đó d_i là bậc của đỉnh i . Theo định lý ma trận cây, bỏ hàng cuối và cột cuối của A để được ma trận B , thì số cây khung bằng $|B|$.

Với đồ thị vòng 8 đỉnh có cạnh giữa các đỉnh cách nhau không quá 2, ma trận A là

$$\begin{pmatrix} 4 & -1 & -1 & 0 & 0 & 0 & -1 & -1 \\ -1 & 4 & -1 & -1 & 0 & 0 & 0 & -1 \\ -1 & -1 & 4 & -1 & -1 & 0 & 0 & 0 \\ 0 & -1 & -1 & 4 & -1 & -1 & 0 & 0 \\ 0 & 0 & -1 & -1 & 4 & -1 & -1 & 0 \\ 0 & 0 & 0 & -1 & -1 & 4 & -1 & -1 \\ -1 & 0 & 0 & 0 & -1 & -1 & 4 & -1 \\ -1 & -1 & 0 & 0 & 0 & -1 & -1 & 4 \end{pmatrix}.$$

Bỏ hàng và cột cuối được

$$\begin{pmatrix} 4 & -1 & -1 & 0 & 0 & 0 & -1 \\ -1 & 4 & -1 & -1 & 0 & 0 & 0 \\ -1 & -1 & 4 & -1 & -1 & 0 & 0 \\ 0 & -1 & -1 & 4 & -1 & -1 & 0 \\ 0 & 0 & -1 & -1 & 4 & -1 & -1 \\ 0 & 0 & 0 & -1 & -1 & 4 & -1 \\ -1 & 0 & 0 & 0 & -1 & -1 & 4 \end{pmatrix}.$$

Số cây khung là định thức của ma trận này, bằng 3528.

Phương pháp chung quá phức tạp để tính trực tiếp. Vì vậy Tiểu Đổng đơn giản hóa đồ thị: cắt bàn tròn tại một vị trí để mọi người tạo thành một đường thẳng, rồi nối các đỉnh có khoảng cách 1 hoặc 2. Với 8 đỉnh, đồ thị là một đường thẳng trong đó mỗi đỉnh nối với hai đỉnh gần nhất về mỗi phía nếu tồn tại. Số cây khung giảm đi nhiều. Nếu nối cả các đỉnh cách nhau 3, cậu chưa biết tính nhanh. Hãy giúp cậu tính số cây khung của loại đồ thị này. Bài toán là Count của NOI 2007.



Hình 3.2: sau khi cắt vòng tròn thành một đường thẳng, mỗi đỉnh chỉ nối tới các đỉnh cách nó không quá 2 vị trí nếu còn tồn tại.

Dữ liệu vào. Tập vào chứa hai số nguyên k, n cách nhau bởi một dấu cách. Tham số k nghĩa là mọi cặp đỉnh có khoảng cách không quá k đều được nối; n là số đỉnh.

Dữ liệu ra. In một số nguyên là số cây khung modulo 65521.

Ví dụ.

Input	Output
3 5	75

Ràng buộc. Với mọi test, $2 \leq k \leq n$.

Test	Miền k	Miền n	Test	Miền k	Miền n
1	$k = 2$	$n \leq 10$	6	$k \leq 5$	$n \leq 100$
2	$k = 3$	$n = 5$	7	$k \leq 3$	$n \leq 2000$
3	$k = 4$	$n \leq 10$	8	$k \leq 5$	$n \leq 10000$
4	$k = 5$	$n = 10$	9	$k \leq 3$	$n \leq 10^{15}$
5	$k \leq 3$	$n \leq 100$	10	$k \leq 5$	$n \leq 10^{15}$

Một cách tính định thức: với $\alpha(P)$ là số cặp nghịch thế trong hoán vị P , ta có

$$|B| = \sum_{P=p_1 p_2 \dots p_n} (-1)^{\alpha(P)} \prod_{j=1}^n b_{j, p_j}.$$

Chẳng hạn

$$B = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 0 \end{pmatrix}.$$

Bảng các hạng tử là

P	$\alpha(P)$	b_{1,p_1}	b_{2,p_2}	b_{3,p_3}	$(-1)^{\alpha(P)} \prod b_{j,p_j}$
123	0	1	5	0	0
132	1	1	6	8	-48
213	1	2	4	0	0
231	2	2	6	7	84
312	2	3	4	8	96
321	3	3	5	7	-105

nên $|B| = 27$.

Phân tích. Trước hết xem việc tìm quy luật có đem lại tính chất quan trọng nào hay không. Với $k = 2$, ký hiệu $f_{n,2}$ là số cây khung khi có n đỉnh. Dãy nhận được là

$$1, 3, 8, 21, 55, \dots$$

Những ai quen Fibonacci sẽ nhận ra đây là các số Fibonacci ở vị trí chẵn:

$$f_{n,2} = \text{Fibonacci}_{2n-2}.$$

Chứng minh phác thảo như sau. Gọi G_n là số cách chia n đỉnh thành hai cây theo yêu cầu, trong đó đỉnh $n-1$ và n không thuộc cùng một cây. Rõ ràng $f_{2,2} = 1$, $G_2 = 1$, và

$$f_{n,2} = 2f_{n-1,2} + G_{n-1}, \quad G_n = G_{n-1} + f_{n-1,2}.$$

Hai công thức này thu được bằng cách xét cách chọn các cạnh nối với đỉnh cuối. Ghép các truy hồi và dùng quy nạp sẽ cho $f_{n,2} = \text{Fibonacci}_{2n-2}$.

Với $k = 3$, dãy bắt đầu là

$$1, 3, 16, 75, 336, 1488, \dots$$

Rất khó nhận ra tính chất đặc biệt; ngay cả tra cứu dãy này trên On-Line Encyclopedia of Integer Sequences thời điểm đó cũng không thấy. Với $k = 4, 5$ tình hình tương tự.

Làm sao tìm được truy hồi? Bắt chước chứng minh của $k = 2$ thì các trường hợp quá rối. Nguyên nhân là các cạnh được thêm để tạo chu trình, khiến ta khó biết trường hợp nào hợp lệ. Vì sao có chu trình? Vì hai đỉnh được nối có thể đã thuộc cùng một thành phần liên thông. Vậy hãy thêm một chiều trạng thái để ghi nhận tính liên thông.

Xét đồ thị N đỉnh, chỉ quan tâm quan hệ liên thông của K đỉnh cuối. Dùng biểu diễn chuẩn nhỏ nhất: hai vị trí có cùng số nghĩa là thuộc cùng một thành phần. Ví dụ với $K = 3$ có 5 trạng thái

$$111, 112, 121, 122, 123.$$

Với $K = 4$ có 15 trạng thái, và với $K = 5$ có 52 trạng thái.

Từ trạng thái của $N - 1$ đỉnh, ta suy ra trạng thái của N đỉnh bằng cách xét các cạnh từ đỉnh cuối về phía trước. Quan hệ này cho phép xây dựng ma trận chuyển cơ sở. Tiền xử lý có thể thực hiện bằng đệ quy.

Tiếp theo tính số cách cho từng trạng thái liên thông khi $N = K$ để làm vector cơ sở, cũng có thể dùng đệ quy. Cuối cùng nhân ma trận: lấy vector cơ sở nhân với ma trận chuyển lũy thừa $N - K$. Đáp án là số cách ở trạng thái mà K đỉnh cuối cùng thuộc cùng một thành phần liên thông.

Độ phức tạp thời gian là

$$O(\log_2 N \cdot 52^3),$$

trong đó 52 là số trạng thái khi $K = 5$. Độ phức tạp bộ nhớ là $O(52^2)$.

Nhận xét. Đến cuối, ta thấy bài toán không chỉ chứa một dãy, mà đáp án nằm trong một trong nhiều dãy được phân biệt bởi trạng thái liên thông. Các dãy này lại có quan hệ qua ma trận chuyển. Lời giải bắt đầu từ quan sát trường hợp $k = 2$, sau đó đi vào truy hồi trạng thái và dùng nhân ma trận để xử lý miền n rất lớn.

3.2 Ví dụ 4: Maximum

Mô tả bài toán. Dãy $\{A_n\}$ thỏa mãn

$$A_1 = 1, \quad A_{2i} = A_i, \quad A_{2i+1} = A_i + A_{i+1}.$$

Cho n , hãy tìm giá trị lớn nhất trong $A_1, A_2, \dots, A_n$. Bài toán là Ural 1396; dãy còn được gọi là Stern's diatomic series hoặc fusc.

Dữ liệu.

$$1 \leq n \leq 10^{18}.$$

Phân tích. Miền N rất lớn, nên cần thuật toán $O(1)$ hoặc $O(\log N)$. Các số hạng đầu là

$$1, 1, 2, 1, 3, 2, 3, 1, 4, 3, 5, 2, 5, 3, 4, 1, 5, 4, 7, 3, \dots$$

Thoạt nhìn không thấy quy luật. Công thức truy hồi liên quan đến 2, nên ta xét theo các đoạn giữa hai lũy thừa của 2:

$$1, \{1\}$$

$$1, 2, \{1\}$$

$$1, 3, 2, 3, \{1\}$$

$$1, 4, 3, 5, 2, 5, 3, 4, \{1\}$$

$$1, 5, 4, 7, 3, 8, 5, 7, 2, 7, 5, 8, 3, 7, 4, 5, \{1\}.$$

Hàng thứ i có thể thu được từ hàng thứ $i - 1$: các vị trí lẻ là số của hàng trước, các vị trí chẵn là tổng của hai vị trí lẻ kề nhau; vị trí chẵn cuối là số lẻ cuối cộng 1. Thực ra kết luận tổng quát hơn cũng đúng:

$$A_{(2i+1)2^k} = A_{i2^k} + A_{(i+1)2^k},$$

và điều này chứng minh được trực tiếp từ định nghĩa dãy.

Quan sát giá trị lớn nhất mỗi hàng:

$$1, \{1\}; \quad 1, 2, \{1\}; \quad 1, 3, 2, 3, \{1\};$$

1, 4, 3, 5, 2, 5, 3, 4, {1};

1, 5, 4, 7, 3, 8, 5, 7, 2, 7, 5, 8, 3, 7, 4, 5, {1}.

Ta thấy giá trị lớn nhất ở hàng thứ i là số Fibonacci thứ $i + 1$, và nó đứng cạnh số Fibonacci thứ i .

Chứng minh: với $i \leq 3$, kiểm tra các số hạng đầu là đủ. Khi $i > 3$, giá trị lớn nhất ở hàng thứ i chỉ có thể là tổng của giá trị lớn nhất hàng $i - 1$ và giá trị lớn nhất hàng $i - 2$, vì các số mới sinh ở hàng $i - 1$ không kề nhau (chúng nằm ở vị trí chẵn), còn các số cũ của hàng $i - 2$ cũng không kề nhau (nằm ở vị trí lẻ). Theo giả thiết quy nạp, hai giá trị ấy là Fibonacci_i và Fibonacci_{i-1} , nên giá trị lớn nhất của hàng i là Fibonacci_{i+1} , đồng thời nó kề với Fibonacci_i .

Tổng quát hơn, nếu hai số kề nhau là a, b với $a < b$, thì sau khi mở rộng n lần, giá trị lớn nhất trong đoạn sinh ra là số hạng thứ $n + 1$ của dãy Fibonacci tổng quát có hai số đầu a, b . Nguyên lý chứng minh tương tự.

Nhờ đó bài toán giải được trong $O(\log_2 N)$ thời gian và bộ nhớ. Cần lưu ý giá trị lớn nhất có thể vượt quá int64 , nên nên dùng số nguyên không dấu 64 bit hoặc số thực đủ chính xác.

```
function DFSSolve(count, left, right, leftValue, rightValue)
  mid <- (left + right) / 2
  midValue <- leftValue + rightValue
  if mid <= N + 1 then
    x <- generalized_fibonacci(count + 1, leftValue, midValue)
    if mid <= N then
      x <- max(x,
               DFSSolve(count - 1, mid, right,
                        midValue, rightValue))
    end if
    return x
  else
    return DFSSolve(count - 1, left, mid, leftValue, midValue)
  end if
end function

procedure Main
  find the minimum integer T such that 2^T > N
  left <- 2^(T - 1)
  right <- 2^T
  ans <- generalized_fibonacci(T, 1, 1)
  ans <- max(ans, DFSSolve(T, left, right, 1, 1))
  print ans
end procedure
```

Một công thức ngắn có thể truyền tải nhiều thông tin hơn về ngoài của nó. Nếu nhìn thấu bản chất và khai thác đúng cách, nó trở thành chìa khóa cho lời giải.

4 Kết luận

Xã hội ngày nay là xã hội thông tin. Công nghệ thông tin được dùng rộng rãi trong sản xuất và đời sống, cung cấp công cụ tính toán mạnh cho nhiều lĩnh vực kỹ thuật và đóng góp lớn cho sự phát triển của các ngành nghề. Từ kinh nghiệm tham gia thi tin học lâu năm, tác giả nhận thức sâu sắc tầm quan trọng của tin học và có những suy nghĩ riêng về môn học này. Trên nền tảng các hoạt động thi đấu, bài viết giới thiệu kiến thức cơ bản, phương pháp thường dùng và tư tưởng phổ biến khi xử lý bài toán dãy số.

Dãy số là một dạng hàm đặc biệt và có ứng dụng rộng trong tin học cũng như bài toán thực tế. Các vấn đề như tốc độ tăng trưởng, tốc độ suy giảm trong sản xuất và đời sống, lãi suất tiền gửi, trả góp, giao dịch kỳ hạn, tăng dân số theo tháng, thường có thể quy về bài toán dãy số.

Trong thi tin học, bài toán dãy số thường kiểm tra khả năng nghiên cứu tính chất số hạng dựa trên công thức truy hồi. Bài viết trước hết cung cấp các kiến thức toán học liên quan, sau đó đưa ra phương pháp tính nhanh số hạng thứ N của dãy truy hồi tuyến tính, và trên cơ sở ấy giới thiệu cách khai thác tính tuần hoàn của dãy. Các phương pháp tư duy và kỹ thuật thường gặp gồm đào sâu bản chất bài toán, tìm công thức số hạng tổng quát, xét theo từng giai đoạn, cũng như khai thác tính chất của một số dãy đặc biệt.

Trong nhiều lĩnh vực như quân sự và kỹ thuật, tính toán bằng máy tính là điều thiết yếu. Trong số các phương pháp thuật toán, phương pháp lặp giữ vị trí quan trọng, mà phương pháp lặp lại gắn chặt với dãy số; vì vậy dãy số càng trở nên đáng chú ý. Cùng với cải cách giáo dục, giáo dục tin học thế kỷ 21 ngày càng coi trọng tính tổng hợp và tính ứng dụng. Bài toán dãy số có đặc điểm linh hoạt, giàu kỹ thuật, đòi hỏi tư duy cao và phân loại năng lực tốt, nên ngày càng được người ra đề ưa chuộng. Người ra đề thường dùng các bài mang tính khám phá, ứng dụng và thú vị để kiểm tra trình độ kiến thức, năng lực tư duy và ý thức sáng tạo của thí sinh.

Giải bài toán dãy số tổng hợp và ứng dụng cần cả nền tảng kiến thức vững chắc lẫn năng lực tư duy logic, phân tích và giải quyết vấn đề. Ta nên vận dụng quan sát, quy nạp, phỏng đoán để xây dựng mô hình cấp số cộng, cấp số nhân hoặc dãy truy hồi, rồi kết hợp kiến thức liên quan khác.

Bài toán dãy số biến hóa rất nhiều. Quy chuyển không phải công cụ vạn năng, nhưng nếu nắm chắc bản chất, quan sát cẩn thận, dám phỏng đoán, có tinh thần sáng tạo và biết phát hiện vẻ đẹp của dãy số, ta có thể vượt qua khó khăn để đến lời giải. Điều này cũng đúng với các bài toán tin học khác. Mong rằng độc giả có thể suy rộng từ các ví dụ, suy nghĩ nhiều hơn, phân tích chăm hơn, mở rộng tư duy và vận dụng tổng hợp các phương pháp trong bài viết này vào nhiều loại bài toán, không chỉ riêng bài toán dãy số.

Tài liệu tham khảo

1. The On-Line Encyclopedia of Integer Sequences, <http://www.research.att.com/~njas/sequences/index.html>.
2. Michael Rybak, thảo luận về Ural 1309, <http://acm.timus.ru/forum/thread.aspx?space=1&num=1309&id=12571>.
3. Hồ Văn Đồng, “Báo cáo lời giải đếm cây khung”, 2007.

Lời cảm ơn

Trước hết, xin cảm ơn giáo viên hướng dẫn Trần Đình.

Đặc biệt cảm ơn huấn luyện viên đội tuyển quốc gia Lưu Nhữ Giai đã đưa ra nhiều ý kiến và đề xuất cho bài viết.

Cảm ơn Trần Đan Kỳ của Trường Nhả Lễ Trường Sa, Du Hoa Trình của Trường Trung học số 2 Hàng Châu và Cố Nghiên của Trường Trung học số 1 Trung Sơn, Quảng Đông. Họ đã cung cấp nhiều ý tưởng tốt cho bài viết, đồng thời nhiệt tình giúp đỡ trong học tập và luyện tập thường ngày, đem lại nhiều gợi ý và kỹ thuật thi đấu.

Cảm ơn Trần Du Hy, Trương Dục Thừa và Trần Khải Phi ở Giang Tô đã cung cấp lời giải tham khảo cho Ural 1396.

Đặc biệt cảm ơn bạn Nhậm Hân Vũ khóa 2008 của Trường Trung học số 1 Phúc Châu. Bạn đã giúp đỡ rất nhiều mặt, chỉnh lý câu chữ cho bài viết trong lúc học tập lớp 12 bận rộn, và đem lại nhiều gợi ý về toán học.

Cảm ơn Hồ Bá Đào khóa 2007; Tiêu Hán Tuấn, Lưu Cần, Đường Kiện Thường và Trương Tâm Trình khóa 2008; Chu Hồng Vũ khóa 2009; Lý Hiểu Tiêu khóa 2010 của Trường Trung học số 1 Phúc Châu.